

លំហាត់តក្កវិជ្ជាទាំង ១០០

បណ្តុំលំហាត់គ្រប់កម្រិត ផ្អែកលើតក្កវិជ្ជា "Logic over Code"

វត្ថុបំណង: កម្រងលំហាត់ទាំង ១០០ នេះត្រូវបានរៀបចំឡើងយ៉ាងយូធីត ដោយផ្ដោតលើ Business Logic, Edge Cases, និង Expected Output ដើម្បីឱ្យនិស្សិតយល់ច្បាស់ពីបញ្ហា និងការគិតរកដំណោះស្រាយ (Problem Solving) មុនពេលសរសេរកូដជាក់ស្តែង។

កម្រិតទី ១: មូលដ្ឋានគ្រឹះ និងប្រមាណវិធី (Variables & Basic Math)

លំហាត់ទី ១: ការប្តូរប្រាក់ (Advanced KHR-USD Exchange)

- Business Logic:** ទទួលបានប្រាក់ USD រួចបំប្លែងទៅជា KHR (អត្រាការប្តូរថេរ \$1 = 4100៛)។ ប៉ុន្តែ បើទឹកប្រាក់ USD បញ្ចូលមកមានតម្លៃតូចជាង ឬស្មើ ០ ត្រូវបង្ហាញសារព្រមានថា 'ទិន្នន័យមិនត្រឹមត្រូវ'។
- Edge Case:** អ្នកប្រើប្រាស់បញ្ចូលលេខទសភាគ (ឧទាហរណ៍: 2.75)។ លទ្ធផលចុងក្រោយជាលុយរៀលត្រូវតែជា ចំនួនគត់ (ទម្លាក់ក្បៀសចោល) ព្រោះលុយរៀលគ្មានកាក់ទសភាគទេ។
- Expected Output:** Input: 2.5 USD -> Output: 10250 Riel

លំហាត់ទី ២: គណនារបៀបប្រាក់សេវាកម្ម (Restaurant Bill with Service Charge)

- Business Logic:** បញ្ចូលតម្លៃម្ហូប ៣ មុខ។ មុននឹងបូកសរុប ត្រូវគិតថ្លៃសេវាកម្ម (Service Charge) 5% លើមុខម្ហូបទី១ និងទី២ ប៉ុន្តែមិនគិតលើមុខម្ហូបទី៣ ទេ (ឧបមាថាជាភេសជ្ជៈ)។ រួចទើបបូកបញ្ចូលគ្នាដើម្បីរកតម្លៃចុងក្រោយ។
- Edge Case:** តម្លៃម្ហូបអាចជាលេខអវិជ្ជមាន ក្នុងករណីបញ្ចូលខុស។ ត្រូវតែពិនិត្យភាពត្រឹមត្រូវ។
- Expected Output:** Input: 10\$, 20\$, 5\$ -> Total: (10*1.05) + (20*1.05) + 5 = 36.5\$

លំហាត់ទី ៣: រកបរិមាត្រ និងផ្ទៃក្រឡាផ្ទះ (Khmer House Dimensions)

- Business Logic:** បញ្ចូលប្រវែងបណ្តោយ និងទទឹងនៃដីសង់ផ្ទះ។ គណនាទំហំបរិមាត្រ និងផ្ទៃក្រឡា។ បង្គ្រប់លក្ខខណ្ឌ៖ ប្រសិនបើបណ្តោយតូចជាងទទឹង ត្រូវផ្លាស់ប្តូរតម្លៃបណ្តោយជាមួយបណ្តោយ (Swap) មុននឹងគណនា ដើម្បីធានាថាបណ្តោយតែងតែធំជាងទទឹង។
- Edge Case:** តម្លៃបណ្តោយជាសូន្យ ឬអវិជ្ជមាន ត្រូវចាត់ទុកជាលេខអវិជ្ជមាន។
- Expected Output:** Input: Length=4, Width=6 -> Swap -> Length=6, Width=4 -> Perimeter=20m, Area=24m²

លំហាត់ទី ៤: មធ្យមភាគពិន្ទុ (Weighted Grading)

- Business Logic:** បញ្ចូលពិន្ទុ ៥ មុខវិជ្ជា។ ប៉ុន្តែមុខវិជ្ជា 'គណិតវិទ្យា' និង 'ភាសាខ្មែរ' មានមេគុណ (Weight) ស្មើ ២ ឯមុខវិជ្ជា ៣ ទៀតមានមេគុណ ១។ គណនាមធ្យមភាគដោយផ្អែកលើមេគុណសរុប (ចែកនឹង ៧)។
- Edge Case:** ពិន្ទុដែលបញ្ចូលត្រូវតែស្ថិតនៅក្នុងចន្លោះពី ០ ដល់ 100 ដាច់ខាត។
- Expected Output:** Math: 90, Khmer: 80, Sci: 70, Eng: 60, His: 50 -> Average = ((90*2)+(80*2)+70+60+50)/7

លំហាត់ទី ៥: ការបែងចែកផ្លែឈើ និងការខូចខាត (Mango Distribution Logic)

- Business Logic:** បញ្ចូលចំនួនផ្លែស្វាយសរុប និងចំនួនសិស្ស។ មុននឹងចែក ត្រូវដក 10% នៃផ្លែស្វាយចេញសិន (ឧបមាថាជាស្វាយរលួយ)។ រួចទើបយកចំនួនស្វាយស្រស់ដែលនៅសល់ មកចែកឱ្យសិស្សស្មើៗគ្នា។ រកចំនួនដែលសិស្សម្នាក់ៗទទួលបាន និងចំនួនដែលនៅសល់ចុងក្រោយ។
- Edge Case:** លទ្ធផលនៃការដក 10% អាចចេញជាលេខក្បៀស ដូចនេះក្នុងករណីនេះប្រើ Math.floor()

ដើម្បីឱ្យចំនួនស្វាយល្អសល់ជាចំនួនគត់មុនចែក។

• **Expected Output:** Input: 100 Mangoes, 3 Students -> Good Mangoes = 90 -> Each gets: 30, Leftover: 0

លំហាត់ទី 6: គណនាអាយុ និងសិទ្ធិការងារ (Age & Work Eligibility)

- **Business Logic:** បញ្ចូលឆ្នាំកំណើត (គ.ស)។ គណនាអាយុបច្ចុប្បន្ន (ផ្អែកលើឆ្នាំ ២០២២)។ បង្ហាញអាយុ និងតម្លៃ Boolean (true/false) បញ្ជាក់ថា តើគាត់ស្ថិតក្នុងអាយុធ្វើការងារបានដែរឬទេ (អាយុចន្លោះពី ១៨ ដល់ ៦០ ឆ្នាំ)។
- **Edge Case:** ប្រសិនបើឆ្នាំកំណើតដែលបញ្ចូល ធំជាងឆ្នាំបច្ចុប្បន្ន (២០២២) កម្មវិធីត្រូវបង្ហាញសារកំហុស។
- **Expected Output:** Input: 2000 -> Age: 26 -> Eligible: true

លំហាត់ទី 7: ប្រព័ន្ធបង្កើតទម្រង់ឈ្មោះខ្មែរ (Khmer Name Formatting)

- **Business Logic:** ទទួលបាននិមិត្តរូប ៣ ដាច់ដោយឡែកពីគ្នា៖ នាមត្រកូល (Last Name), នាមខ្លួន (First Name), និង ភេទ (Gender: 'M' ឬ 'F')។ កម្មវិធីត្រូវផ្តល់ឈ្មោះពេញលេញ បើភេទ 'M' ត្រូវថែមពាក្យ 'លោក' ពីមុខ បើ 'F' ត្រូវថែមពាក្យ 'កញ្ញា/លោកស្រី'។
- **Edge Case:** អ្នកប្រើប្រាស់អាចវាយដកឃ្លាខុសបច្ចេកទេស ដូចនេះត្រូវប្រើប្រាស់ Method .trim() ដើម្បីសម្អាតនិមិត្តរូប។
- **Expected Output:** Input: 'សុខ', 'កំសាន្ត', 'M' -> Output: 'ស្ដេច លោក សុខ កំសាន្ត'

លំហាត់ទី 8: គណនាល្បឿន និងការលឿនសម្រាក (Travel Speed Calculator)

- **Business Logic:** គណនាល្បឿនមធ្យម (គីឡូម៉ែត្រ/ម៉ោង) ពីភ្នំពេញទៅសៀមរាប (ចម្ងាយ 315 គ.ម)។ អ្នកប្រើប្រាស់បញ្ចូលរយៈពេលធ្វើដំណើរសរុប (ជាម៉ោង) ប៉ុន្តែ Logic ត្រូវដកថយ ៣០ នាទី (0.5 ម៉ោង) ចេញពីម៉ោងសរុបសិន ព្រោះជាពេលវេលាលឿនសម្រាកប្រចាំថ្ងៃ។
- **Edge Case:** ប្រសិនបើរយៈពេលដែលបញ្ចូលមកតូចជាង ឬស្មើ 0.5 ម៉ោង វានឹងឱ្យម៉ោងបើកបរទៅជាអវិជ្ជមាន ឬសូន្យ ដែលបង្កឱ្យមានកំហុស ArithmeticException (ការចែកនឹងសូន្យ)។
- **Expected Output:** Input: Total Time = 5.5 hours -> Driving Time = 5.0 hours -> Speed = $315/5 = 63$ km/h

លំហាត់ទី 9: ប្តូរសីតុណ្ហភាពសម្រាប់រាយការណ៍ព័ត៌មាន (Celsius to Fahrenheit & Kelvin)

- **Business Logic:** បញ្ចូលសីតុណ្ហភាពជា Celsius។ គណនាបំលែងទៅជា Fahrenheit ផង និង Kelvin ផង រួចបង្ហាញលទ្ធផលទាំងពីរក្នុងពេលតែមួយ។
- **Edge Case:** តម្លៃទាបបំផុតនៃសីតុណ្ហភាពក្នុងចក្ខុវិស័យគឺ Absolute Zero (-273.15°C) បើបញ្ចូលទាបជាងនេះ ត្រូវប្រកាសថាទិន្នន័យមិនអាចទៀងទាត់។
- **Expected Output:** Input: 30°C -> Output: 86°F and 303.15K

លំហាត់ទី 10: គណនាផ្ទៃក្រឡាដីទ្វីត្រីកោណ (Triangular Land Area)

- **Business Logic:** គណនាផ្ទៃក្រឡាដីទ្វីត្រីកោណ បញ្ចូលប្រវែងបាត និងកម្ពស់។ បន្ថែម Business Rule: ប្រសិនបើផ្ទៃក្រឡាគណនាឃើញលើសពី ១០០ ម៉ែត្រការ៉េ ត្រូវគិតថ្លៃពន្ធដី 2\$ ក្នុងមួយម៉ែត្រការ៉េ បើទាបជាងនេះមិនគិតពន្ធឡើយ។
- **Edge Case:** ការគណនាផ្ទៃក្រឡាត្រីកោណត្រូវប្រាកដថាប្រើប្រាស់ទម្រង់បំណែក (floating-point division 1.0/2.0) ដើម្បីកុំឱ្យបាត់បង់តម្លៃក្បៀស។
- **Expected Output:** Input: Base=20m, Height=15m -> Area=150m² -> Tax=300\$

លំហាត់ទី 11: គណនា BMI និងទម្ងន់ស្តង់ដារ (Advanced BMI)

- **Business Logic:** បញ្ចូលទម្ងន់ (kg) និងកម្ពស់ (m) រួចគណនា BMI។ បន្ថែម Logic គណនា 'ទម្ងន់ស្តង់ដារតាមទ្រឹស្តី' (Ideal Body Weight) សម្រាប់កម្ពស់នោះ ដោយប្រើរូបមន្តសាមញ្ញ៖ Ideal Weight = (កម្ពស់ជាសង់ទីម៉ែត្រ - ១០០) * ០.៩។
- **Edge Case:** កម្ពស់ត្រូវតែគិតជាម៉ែត្រក្នុងការរក BMI ប៉ុន្តែគិតជាសង់ទីម៉ែត្រក្នុងការរក Ideal Weight (ត្រូវធ្វើប្រមាណវិធីបំលែង x100)។
- **Expected Output:** Input: Weight=70kg, Height=1.7m -> BMI=24.22, Ideal Weight=63kg

លំហាត់ទី 12: គណនាការប្រាក់ប្រកួសមាស (Compound Interest Simulation)

- **Business Logic:** សិស្សភាគច្រើនស្គាល់តែការប្រាក់សាមញ្ញ។ លំហាត់នេះតម្រូវឱ្យគណនាការប្រាក់សមាស (Compound Interest) រយៈពេល ២ ឆ្នាំ (ការប្រាក់ឆ្នាំទី១ បូកចូលដើមសម្រាប់គិតឆ្នាំទី២)។ បញ្ចូលប្រាក់ដើម និងអត្រាការប្រាក់ប្រចាំឆ្នាំ។
- **Edge Case:** លទ្ធផលទឹកប្រាក់ត្រូវ Format ឱ្យបង្ហាញត្រឹមតែ ២ ខ្ទង់ក្រោយក្បៀសប៉ុណ្ណោះ (ឧទាហរណ៍ប្រើ `String.format("%.2f")`)។
- **Expected Output:** Input: Principal=1000\$, Rate=10% -> Year 1: 1100\$ -> Year 2 End Total: 1210\$

លំហាត់ទី 13: ប្រព័ន្ធបំបែកពេលវេលាដោយប្រើប្រាស់ (Download Time Splitter)

- **Business Logic:** បញ្ចូលរយៈពេលសរុបជា 'វិនាទី' (ទិន្នន័យឆ្នាំ ឧទាហរណ៍: ៩៥០០ វិនាទី)។ គណនាបំបែកវាទៅជាម៉ោង ម៉ោងនោះវិនាទី ដោយប្រើការចែកដាច់ និងការរកសំណល់ ជាមួយលេខ ៣៦០០ និង ៦០។
- **Edge Case:** បើបញ្ចូលចំនួនវិនាទីតូចជាង ៦០ នោះផ្ទេរម៉ោង និងនាទីត្រូវតែបង្ហាញលេខ ០ មិនមែនមិនបង្ហាញនោះទេ។
- **Expected Output:** Input: 3665 seconds -> Output: 1 Hour, 1 Minute, 5 Seconds

លំហាត់ទី 14: គណនាមាឌទឹកស៊ីឡាំង (Cylinder Water Tank Capacity)

- **Business Logic:** គណនាមាឌទឹកស៊ីឡាំង (ប្រើ `Math.PI`)។ បញ្ចូលកាំ r និងកម្ពស់ h ជាម៉ែត្រ។ បន្ថែម Logic បំប្លែងពីម៉ែត្រគូប ទៅជា លីត្រ (១ ម៉ែត្រគូប = ១០០០ លីត្រ)។
- **Edge Case:** តម្លៃ `Math.PI` មានទសភាគច្រើន ដូចនេះលទ្ធផលចុងក្រោយត្រូវបំប្លែងជាចំនួនគត់ ឬកាត់ក្បៀសដើម្បីងាយស្រួលអានលំដាប់លីត្រ។
- **Expected Output:** Input: $r=1, h=2$ -> Volume $\approx 6.28 \text{ m}^3$ -> Capacity = 6283 Liters

លំហាត់ទី 15: ផលធៀបភេទក្នុងក្រុម (Gender Ratio Logic)

- **Business Logic:** បញ្ចូលចំនួនសិស្សប្រុស និងចំនួនសិស្សស្រី។ គណនាមេរ័ត្រ ១. ចំនួនសិស្សសរុប, ២. ភាគរយសិស្សស្រី, ៣. ផលធៀបសិស្សប្រុសធៀបនឹងសិស្សស្រី (Male to Female Ratio)។
- **Edge Case:** បើចំនួនសិស្សស្រីបញ្ចូលមកស្មើ ០ ក្នុងត្រូវទប់ស្កាត់ការចែកនឹងសូន្យ (ArithmeticException) ដោយប្រើលក្ខខណ្ឌការពារជាមុន។
- **Expected Output:** Input: Male=30, Female=10 -> Total=40, Female=25%, Ratio=3:1

លំហាត់ទី 16: គណនាតម្លៃបញ្ចុះពីសេសទ្វេដង (Double Discount Logic)

- **Business Logic:** បញ្ចូលតម្លៃទំនិញដើម។ អនុវត្តការបញ្ចុះតម្លៃលើកទី១ ចំនួន 10%។ បន្ទាប់មក យកតម្លៃដែលបញ្ចុះរួចនោះមកបញ្ចុះបន្ថែម 5% ទៀត (Coupon discount)។ បង្ហាញតម្លៃចុងក្រោយ និងទឹកប្រាក់សរុបដែលបានសន្សំ។
- **Edge Case:** ការបញ្ចុះតម្លៃទ្វេដង (10% + 5%) មិនស្មើនឹងការបញ្ចុះតម្លៃ 15% ម្តងនោះទេ (នេះជាគុណវិបាកនៃការបញ្ចុះតម្លៃលើលំដាប់)។
- **Expected Output:** Input: 100\$ -> First: 90\$ -> Second: 85.5\$ -> Total Saved: 14.5\$

លំហាត់ទី 17: ប្រព័ន្ធបង្កើត Subnet Mask ងាយស្រួល (IP & Subnet Builder)

- **Business Logic:** ទទួលតម្លៃលេខកូដបណ្តាញ IP ៤ ខ្ទង់ដាច់ពីគ្នា និងតម្លៃ Prefix (ឧទាហរណ៍: ២៤)។ ភារកិច្ចគឺបង្ហាញ IP នោះ រួមទាំងសរសេរលក្ខខណ្ឌងាយមួយ៖ បើ Prefix ស្មើ ២៤ ឱ្យបង្ហាញ Subnet Mask: 255.255.255.0 បើ Prefix ស្មើ ១៦ ឱ្យបង្ហាញ 255.255.0.0។
- **Edge Case:** រាល់ខ្ទង់នីមួយៗនៃ IP (Octet) ត្រូវតែស្ថិតនៅក្នុងចន្លោះពី ០ ដល់ 255 ដាច់ខាត។
- **Expected Output:** Input: 192, 168, 1, 1, Prefix=24 -> Output: 192.168.1.1/24 (Mask: 255.255.255.0)

លំហាត់ទី 18: គណនាតម្លៃទំនិញរួមបញ្ចូលពន្ធអាករ (VAT Layering)

- **Business Logic:** បញ្ចូលតម្លៃទំនិញមិនទាន់គិតពន្ធ។ ប្រព័ន្ធត្រូវគណនាពន្ធពីរដ្ឋណាកំណត់ ពន្ធសេវាកម្មកម្ពុជា ៣% មុន រួចទើបយកតម្លៃនោះមកគិតពន្ធ VAT 10% បន្ថែមពីលើ។ បង្ហាញតម្លៃចុងក្រោយបង្អស់។
- **Edge Case:** ពន្ធ VAT ត្រូវតែគណនាផ្អែកលើតម្លៃដែលបានបូកបញ្ចូលពន្ធសេវាកម្មកម្ពុជារួចរាល់ហើយ (Compound Taxation Rule)។
- **Expected Output:** Input: 100\$ -> Plus 3%: 103\$ -> Plus 10% VAT: 113.3\$

លំហាត់ទី 19: គណនាចម្ងាយរវាងបង្គោលអង្កត់ទ្វី (2D Coordinate Distance)

- **Business Logic:** បញ្ចូលកូអរដោនេ (x1, y1) នៃបង្គោលអង្កត់ទ្វីមួយ និង (x2, y2) នៃបង្គោលទីពីរ។ គណនាចម្ងាយខ្សែចម្លងរវាងបង្គោលទាំងពីរ។
- **Edge Case:** តម្លៃកូអរដោនេអាចជាលេខអវិជ្ជមាននៅក្នុងប្លង់ Cartesian ប្រព័ន្ធត្រូវតែគណនាបានត្រឹមត្រូវជាទីតាំងស្វ័យគុណពីរលុបសញ្ញាដក។
- **Expected Output:** Input: (0,0) and (3,4) -> Distance = 5.0m

លំហាត់ទី 20: គណនាផលចំណេញសុទ្ធប្រចាំខែ (Net Profit & Tax Run)

- **Business Logic:** បញ្ចូលចំណូលសរុប (Revenue) និងការចំណាយសរុប (Expenses) នៃហាងអាជីវកម្មខ្នាតតូច។ គណនាប្រាក់ចំណេញដុល (Gross Profit)។ ប្រសិនបើចំណេញដុលនោះវិជ្ជមាន ត្រូវដកពន្ធអាករជីវកម្ម ១% ចេញ រួចទើបបង្ហាញប្រាក់ចំណេញសុទ្ធ (Net Profit)។
- **Edge Case:** ប្រសិនបើការចំណាយធំជាងចំណូល (ខាតបង់) នោះមិនត្រូវគិតពន្ធ ១% ឡើយ ពោលគឺប្រាក់ខាតបង់ត្រូវរក្សាទុកដដែល។
- **Expected Output:** Input: Rev=5000, Exp=3000 -> Gross=2000 -> Tax=20 -> Net Profit=1980\$

កម្រិតទី ២: លក្ខខណ្ឌ និងការសម្រេចចិត្ត (Control Flow - If-Else & Switch)

លំហាត់ទី 21: ពិនិត្យលទ្ធផលប្រឡង (Exam Grading Pass/Fail)

- **Business Logic:** បញ្ចូលពិន្ទុសរុប បើចាប់ពី ៥០ ឡើងទៅឱ្យជាប់ (Passed) បើមិនដូច្នោះទេឱ្យធ្លាក់ (Failed)។
- **Edge Case:** ប្រសិនបើបញ្ចូលពិន្ទុលើសពី ១០០ ឬទាបជាង ០ ត្រូវបង្ហាញសារថា 'ពិន្ទុមិនត្រឹមត្រូវ'។
- **Expected Output:** Input: 45 -> Output: Failed | Input: 75 -> Output: Passed

លំហាត់ទី 22: ចំណាត់ថ្នាក់និទ្ទេស MoEYS (Ministry Grade Standards)

- **Business Logic:** បញ្ចូលមធ្យមភាគពិន្ទុ រួចបង្ហាញនិទ្ទេស (A, B, C, D, E, F) តាមកម្រិតកំណត់ផ្លូវការរបស់ក្រសួងអប់រំ។
- **Edge Case:** ការប្រើប្រាស់ If-Else If ត្រូវរៀបចំលំដាប់ពីខ្ពស់ទៅទាបឱ្យបានត្រឹមត្រូវដើម្បីកុំឱ្យជាន់លក្ខខណ្ឌ។
- **Expected Output:** Input: 85.5 -> Output: Grade B

លំហាត់ទី 23: គណនាថ្លៃអគ្គិសនី EDC (Tiered Electricity Billing)

- **Business Logic:** គណនាថ្លៃភ្លើងតាមកំរិតជំរិត៖ ១-៥០ គីឡូវ៉ាត់តម្លៃ ៣៥០៛/គីឡូ, ៥១-២០០ តម្លៃ ៧២០៛/គីឡូ, លើសពី ២០០ តម្លៃ ៩៦០៛/គីឡូ។
- **Edge Case:** ត្រូវគិតថ្លៃជាដំណាក់កាលៗ (Progressive calculation) មិនមែនយកតម្លៃចុងក្រោយទៅគុណទាំងអស់នោះទេ។
- **Expected Output:** Input: 120 kWh -> Total: (50*350) + (70*720) = 17500 + 50400 = 67900 Riel

លំហាត់ទី 24: តម្លៃសេវាដឹកជញ្ជូនតាមខេត្ត (Logistics Shipping Fee via Switch)

- **Business Logic:** ទទួលបានខេត្តពីអ្នកប្រើប្រាស់ (Siem Reap, Siem Reap, Battambang) រួចបង្ហាញតម្លៃសេវាដឹកជញ្ជូនពីក្នុងតារាងដោយប្រើ Switch Case។
- **Edge Case:** ត្រូវប្រើប្រាស់ Method .toLowerCase() ឬ .toUpperCase() ដើម្បីទប់ស្កាត់ការបញ្ចូលអក្សរតូចធំខុសគ្នា។
- **Expected Output:** Input: 'Siem Reap' -> Output: Shipping Fee: \$3.00

លំហាត់ទី 25: ពិនិត្យឆ្នាំបង្កប់ (Leap Year Evaluation)

- **Business Logic:** ពិនិត្យតាមលក្ខណៈដែលបញ្ចូលជាឆ្នាំដែលមាន ៣៦៦ ថ្ងៃ។ លក្ខខណ្ឌ: ចែកដាច់នឹង ៤ និងមិនដាច់នឹង ១០០ លុះត្រាតែចែកដាច់នឹង ៤០០។
- **Edge Case:** ត្រូវប្រើប្រាស់ Logical Operators (&&, ||) បញ្ចូលគ្នាក្នុងលក្ខខណ្ឌតែមួយឱ្យមានប្រសិទ្ធភាព។
- **Expected Output:** Input: 2024 -> Output: Leap Year | Input: 2100 -> Output: Not a Leap Year

លំហាត់ទី 26: ប្រព័ន្ធគ្រប់គ្រងពិនិត្យ ATM PIN (ATM Security Authentication)

- **Business Logic:** ត្រួតពិនិត្យលេខកូដ PIN ៤ ខ្ទង់របស់កាត ATM បើខុសលើសពី ៣ ដង ឱ្យចាក់សោគណនី (Account Locked Out)។
- **Edge Case:** ត្រូវមានអថេរ (Counter) ដើម្បីកត់ត្រាចំនួនដងដែលអ្នកប្រើប្រាស់បានវាយខុស។
- **Expected Output:** Attempt 3 False -> Output: Your account has been temporarily locked.

លំហាត់ទី 27: ការបញ្ចុះតម្លៃតាមលក្ខខណ្ឌទិញ (Volume-based E-Commerce Discount)

- **Business Logic:** បើអតិថិជនទិញទំនិញអស់លើសពី ២០\$ ទទួលបានការបញ្ចុះតម្លៃ ១០% បើលើសពី ៥០\$ ទទួលបាន ២០%។
- **Edge Case:** លក្ខខណ្ឌពិនិត្យតម្លៃទំនិញ (លើសពី ៥០\$) ត្រូវតែដាក់នៅខាងលើគេបង្អស់ក្នុងរចនាសម្ព័ន្ធ If-Else។
- **Expected Output:** Input: \$60 -> Discount: 20% -> Final Pay: \$48

លំហាត់ទី 28: ពិនិត្យលេខគូ-សេស និងលេខសូន្យ (Parity & Zero Check)

- **Business Logic:** ទទួលយកចំនួនគត់មួយ រួចប្រាប់ថាវាជាលេខគូ (Even), លេខសេស (Odd), ឬជាលេខសូន្យ (Zero)។
- **Edge Case:** លេខសូន្យចែកនឹង ២ សំណល់សូន្យដែរ ដូចនេះត្រូវតែកត់លក្ខខណ្ឌលេខសូន្យមុនគេបង្អស់។
- **Expected Output:** Input: 0 -> Output: Zero | Input: -5 -> Output: Negative Odd Number

លំហាត់ទី 29: អ្នកបកប្រែថ្ងៃក្នុងសប្តាហ៍ (Weekday Translator)

- **Business Logic:** បញ្ចូលលេខ ១ ដល់ ៧ រួចបង្ហាញឈ្មោះថ្ងៃតាមលេខ (១ -> ច័ន្ទ, ៧ -> អាទិត្យ)។
- **Edge Case:** ករណីបញ្ចូលលេខក្រៅពី ១ ដល់ ៧ ត្រូវប្រើប្រាស់លក្ខខណ្ឌ 'default' ក្នុង Switch Case ដើម្បីព្រមាន (គួរ ប្រយ័ត្នចំពោះលេខអវិជ្ជមានផងដែរ)។
- **Expected Output:** Input: 3 -> Output: ថ្ងៃពុធ | Input: 9 -> Output: លេខមិនត្រឹមត្រូវ

លំហាត់ទី 30: ការណែនាំទំហំអាវតាមគីឡូ (T-Shirt Size Recommendation)

- **Business Logic:** បញ្ចូលទម្ងន់ (គីឡូក្រាម) រួចណែនាំទំហំអាវដែលសមស្រប: ក្រោម ៥០គីឡូ (S), ៥០-៦៥គីឡូ (M), ៦៦-៨០គីឡូ (L), លើសពី ៨០គីឡូ (XL)។
- **Edge Case:** ការប្រើប្រាស់ដែនកំណត់លក្ខខណ្ឌ (Boundary conditions) ត្រូវប្រាកដថាមានចន្លោះគីឡូណាមួយដែលរំលងឡើយ។
- **Expected Output:** Input: 65 kg -> Output: Recommended Size: M

លំហាត់ទី 31: ពិនិត្យអាយុបោះឆ្នោតជាតិ (National Election Eligibility)

- **Business Logic:** ពិនិត្យមើលថា តើពលរដ្ឋមានអាយុគ្រប់គ្រាន់ (១៨ ឆ្នាំឡើង) និងមានសញ្ជាតិខ្មែរ (boolean) ទើបមានសិទ្ធិបោះឆ្នោត។
- **Edge Case:** ត្រូវប្រើប្រាស់លក្ខខណ្ឌបញ្ចូលគ្នាដោយប្រើ Operator `&&`។
- **Expected Output:** Input: Age=17, IsKhmer=true -> Output: Not Eligible to Vote

លំហាត់ទី 32: រកលេខធំបំផុតក្នុងចំណោម ៣ លេខ (Find Maximum of Three Numbers)

- **Business Logic:** បញ្ចូលលេខ ៣ ខុសៗគ្នា រួចប្រើលក្ខខណ្ឌដើម្បីរកលេខណាដែលធំជាងគេបំផុតដោយមិនប្រើប្រាស់ Math.max()។
- **Expected Output:** Input: 15, 25, 20 -> Output: Maximum is 25

លំហាត់ទី 33: ពិនិត្យប្រភេទអក្សរ (Character Type Classifier)

- **Business Logic:** ទទួលអក្សរមួយពី Keyboard រួចប្រាប់ថាវាជា អក្សរធំ, អក្សរតូច ឬជាលេខគត់។
- **Edge Case:** ក្នុងត្រូវប្រើប្រាស់លក្ខខណ្ឌពិនិត្យចន្លោះតម្លៃ Char ផ្ទាល់ (ឧ: 'ch' >= 'A' && ch <= 'Z')។
- **Expected Output:** Input: 'g' -> Output: Lowercase Letter

លំហាត់ទី 34: ប្រព័ន្ធពិនិត្យល្បឿនផ្លូវលឿន (Expressway Speed Limit Control)

- **Business Logic:** បើបើកបរល្បឿនលើសពី ១២០ គីឡូម៉ែត្រ/ម៉ោង ត្រូវបង្ហាញសារពិន័យ បើទាបជាង ៦០ គីឡូម៉ែត្រ/ម៉ោង ព្រមានពីគ្រោះថ្នាក់។
- **Edge Case:** ត្រូវបង្កើត Logic ឱ្យដឹងថាបើល្បឿនចន្លោះពី ៦០ ទៅ ១២០ គឺមានស្ថានភាពមានសុវត្ថិភាព (Normal)។
- **Expected Output:** Input: 130 km/h -> Output: Overspeeding Fine! Penalty Applicable.

លំហាត់ទី 35: ពិនិត្យគណនីសកម្ម និងសមតុល្យ (Bank Account Withdrawal Validation)

- **Business Logic:** ត្រួតពិនិត្យលក្ខខណ្ឌពីរ៖ គណនីត្រូវតែមានស្ថានភាពសកម្ម (isActive == true) និងមានសមតុល្យលុយលើសពីចំនួនដែលត្រូវដក ទើបអនុញ្ញាតឱ្យដកលុយ។
- **Edge Case:** បើដកលុយបាន ត្រូវបង្ហាញសមតុល្យដែលនៅសល់ចុងក្រោយ បើមិនបានត្រូវប្រាប់មូលហេតុច្បាស់លាស់។
- **Expected Output:** Input: Balance=50\$, Withdraw=60\$ -> Output: Transaction Denied. Insufficient Funds.

លំហាត់ទី 36: ការកំណត់តម្លៃសំបុត្រកូនតាមអាយុ (Cinema Ticket Tier Pricing)

- **Business Logic:** បញ្ចូលអាយុរបស់អ្នកទិញ បើក្មេងក្រោម ១២ ឆ្នាំតម្លៃ ៣\$, មនុស្សពេញវ័យ (១២-៦០ឆ្នាំ) ៥\$, និងមនុស្សចាស់ជរា (លើសពី ៦០ឆ្នាំ) ២\$។
- **Edge Case:** ត្រូវប្រើប្រាស់ Nested If ឬ If-Else If ឱ្យបានត្រឹមត្រូវតាមដែនអាយុ។
- **Expected Output:** Input: 65 -> Output: Senior Ticket Price: \$2

លំហាត់ទី 37: ពិនិត្យលទ្ធភាពបង្កើតត្រីកោណ (Triangle Validity Check)

- **Business Logic:** បញ្ចូលមុំ ៣ នៃរូបរាងមួយ រួចពិនិត្យថាតើវាអាចបង្កើតជាត្រីកោណបានដែរឬទេ (ផលបូកមុំទាំង ៣ = ១៨០ ដឺក្រេ និងគ្មានមុំណាមួយស្មើ ០ ឬអវិជ្ជមានឡើយ)។
- **Edge Case:** ត្រូវទប់ស្កាត់ករណីបញ្ចូលមុំអវិជ្ជមាន ទោះបីជាបូកបញ្ចូលគ្នាឃើញ ១៨០ ក៏ដោយ (ឧ: 200, -20, 0)។
- **Expected Output:** Input: 60, 60, 60 -> Output: Valid Triangle

លំហាត់ទី 38: ពិនិត្យស្ថានភាពសីតុណ្ហភាពទឹក (Water State Identifier)

- **Business Logic:** បញ្ចូលសីតុណ្ហភាពទឹកជា Celsius រួចប្រាប់ថាទឹកស្ថិតក្នុងស្ថានភាពជា៖ កក (Solid/Ice) បើតូចជាង ឬស្មើ 0, រាវ (Liquid) បើចន្លោះ 1-99, ឬ ឥដ្ឋ/ចំហាយ (Gas/Steam) បើតូចជាងឬស្មើ 100។

- **Edge Case:** ចំណុចព្រំដែន (0°C និង 100°C) ត្រូវតែបញ្ចូលក្នុងលក្ខខណ្ឌឱ្យច្បាស់លាស់។
- **Expected Output:** Input: 105 -> Output: Steam / Gas State

លំហាត់ទី 39: ប្រព័ន្ធបញ្ជាក់ការចូលតាមតួនាទី (Role-Based Access Control - RBAC)

- **Business Logic:** បញ្ចូល Role របស់អ្នកប្រើប្រាស់ (Admin, Staff, Scholar) រួចបង្ហាញសិទ្ធិចូលប្រព័ន្ធខុសៗគ្នា (Full Access, Edit Access, View Only)។
- **Edge Case:** ករណីវាយពាក្យខុសពី Role ទាំងបី ត្រូវបដិសេធការចូល (Access Denied)។ ជាពិសេសត្រូវធ្វើការឆែកទៅលើការវាយបញ្ចូល space នៅសល់ក្នុងករណីមាន។
- **Expected Output:** Input: 'Scholar' -> Output: Welcome! You have View-Only Access.

លំហាត់ទី 40: ម៉ាស៊ីនគិតលេខសាមញ្ញ (Simple Calculator via Switch)

- **Business Logic:** បញ្ចូលលេខ ២ និងសញ្ញាគណិតវិទ្យា (+, -, *, /) រួចគណនាតាមសញ្ញានោះដោយប្រើ Switch Case។
- **Edge Case:** ក្នុងករណីសញ្ញាចែក (/) ត្រូវឆែកលក្ខខណ្ឌការពារកុំឱ្យលេខទី២ ស្មើ ០ ដាច់ខាត។
- **Expected Output:** Input: 10, 0, '/' -> Output: Cannot divide by zero!

កម្រិតទី ៣: ការប្រើប្រាស់រង្វិលជុំ (Loops & Iterations)

លំហាត់ទី 41: មេលេខខ្មែរ (Multiplication Table Generator)

- **Business Logic:** បញ្ចូលលេខមួយ (ឧទាហរណ៍: លេខ ៥) រួចបង្ហាញមេលេខនោះពី 5×1 រហូតដល់ 5×10 ក្នុង Console។
- **Edge Case:** ទម្រង់បង្ហាញត្រូវរៀបចំជូរឱ្យស្អាត និងមានរបៀបរៀបរយ។
- **Expected Output:** Input: 5 ->
Output: 5 x 1 = 5
...
5 x 10 = 50

លំហាត់ទី 42: រាបថយក្រោយចូលឆ្នាំថ្មី (New Year Countdown Clock)

- **Business Logic:** បង្កើតកម្មវិធីរាបថយក្រោយពី ១០ ដល់ ១ សម្រាប់ពិធីឆ្លងឆ្នាំសកល ដើម្បីឱ្យអ្នកម្តងមួយវិនាទីពិតប្រាកដ។
- **Edge Case:** ត្រូវគ្រប់គ្រង Exception ឱ្យបានត្រឹមត្រូវក្នុងពេលប្រើប្រាស់។
- **Expected Output:** Output: 10... 9... 8... 1... Happy New Year!

លំហាត់ទី 43: ផលបូកចំនួនកើន (Sum of N Natural Numbers)

- **Business Logic:** គណនាផលបូកនៃលេខគត់ចាប់ពី ១ ដល់ N (N ជាតម្លៃបញ្ចូលពី Keyboard)។
- **Edge Case:** ប្រសិនបើ N ជាលេខអវិជ្ជមាន ត្រូវប្រាប់ឱ្យអ្នកប្រើបញ្ចូលឡើងវិញ (ទាមទារលេខវិជ្ជមាន)។
- **Expected Output:** Input: 5 -> Output: Sum (1+2+3+4+5) = 15

លំហាត់ទី 44: គូររូបត្រីកោណធ្មោល (Right Triangle Pattern)

- **Business Logic:** ប្រើ Nested Loop (Loop ក្នុង Loop) ដើម្បីគូររូបរាងត្រីកោណកែងដោយប្រើតួអក្សរ '*' ផ្អែកលើចំនួនជួរដែលបញ្ចូល។
- **Edge Case:** Loop ខាងក្នុងត្រូវមានលក្ខខណ្ឌអាស្រ័យនឹងតម្លៃ Counter របស់ Loop ខាងក្រៅ។
- **Expected Output:** Input: 3 -> Output:
*
**

លំហាត់ទី 45: ស្វែងរកលេខធំបំផុតពីការបញ្ចូល (Interactive Max Finder)

- **Business Logic:** ឱ្យអ្នកប្រើប្រាស់បញ្ចូលលេខ ១០ ដង (ម្តងមួយៗ) រួចបង្ហាញលេខណាដែលធំជាងគេបំផុតដែលគាត់បានបញ្ចូលចុងក្រោយ។
- **Edge Case:** ក្នុងត្រូវចាប់ផ្តើមប្រៀបធៀបដោយយកលេខដំបូងគេបង្អស់ជា Max មិនមែនចាប់ផ្តើមពីលេខ ០ ទេ។
- **Expected Output:** Input: 5, -2, 20, 3... -> Output: Maximum entered is 20

លំហាត់ទី 46: ពិនិត្យលេខបឋម (Prime Number Verification)

- **Business Logic:** ពិនិត្យថា លេខគត់វិជ្ជមានមួយជាលេខបឋម (ចែកដាច់នឹង ១ និងខ្លួនឯងប៉ុណ្ណោះ)។
- **Edge Case:** ដើម្បីបង្កើនល្បឿន Algorithm Loop គួរតែត្រឹមតែប្រើសកម្មភាពនៃលេខនោះបានហើយ ($i \leq \text{Math.sqrt}(\text{num})$)។
- **Expected Output:** Input: 11 -> Output: 11 is a Prime Number

លំហាត់ទី 47: គណនា Factorial (Factorial Loop Computation)

- **Business Logic:** គណនា $N!$ (ឧទាហរណ៍: $5! = 5 * 4 * 3 * 2 * 1 = 120$)។
- **Edge Case:** $0!$ គឺស្មើ ១ ដូចនេះក្នុងករណីស្ថានភាពលក្ខខណ្ឌពិសេសនេះ។
- **Expected Output:** Input: 5 -> Output: $5! = 120$

លំហាត់ទី 48: លំដាប់លេខ Fibonacci (Fibonacci Series Display)

- **Business Logic:** បង្ហាញលំដាប់លេខ Fibonacci ឱ្យបាន N ចំនួនដំបូង (ចាប់ផ្តើមពី ០, ១, ១, ២, ៣, ៥...)។
- **Edge Case:** ត្រូវប្រើប្រាស់អថេរចំនួនបីដើម្បីផ្លាស់ប្តូរតម្លៃ និងបូកបន្ថែមបន្ទាប់គ្នាក្នុង Loop។
- **Expected Output:** Input: 5 -> Output: 0, 1, 1, 2, 3

លំហាត់ទី 49: ហ្គេមទាយលេខសម្ងាត់ (Number Guessing Game with Break)

- **Business Logic:** កម្មវិធីបង្កើតលេខចៃដន្យ (Random) មួយចន្លោះពី ១ ដល់ ១០០ រួចឱ្យអ្នកប្រើទាយ។ បើទាយត្រឹមត្រូវ ឬធ្វើបាន ត្រូវប្រាប់គាត់ រហូតដល់ត្រូវ ឬអស់ឱកាស ៥ដង។
- **Edge Case:** ប្រើប្រាស់លក្ខខណ្ឌ 'break' ដើម្បីបញ្ចប់ Loop ភ្លាមៗនៅពេលអ្នកប្រើទាយត្រឹមត្រូវ។
- **Expected Output:** Output: Too high! Try again. -> Output: Correct! You won.

លំហាត់ទី 50: បញ្ច្រាស់លេខគត់ (Number Reversal Algorithm)

- **Business Logic:** បញ្ចូលលេខ 12345 រួចប្រើ Loop ដើម្បីបែកទាញយកសំណល់និងបង្ហាញត្រឡប់មកវិញជា 54321។
- **Edge Case:** លទ្ធផលចុងក្រោយត្រូវតែរក្សាទុកក្នុងអថេរប្រភេទលេខគត់ថ្មីមួយ។
- **Expected Output:** Input: 1234 -> Output: Reversed Number: 4321

លំហាត់ទី 51: រាប់លេខសេស (Odd Number Counter in Range)

- **Business Logic:** បញ្ចូលលេខចាប់ផ្តើម (Start) និងបញ្ចប់ (End) រួចបង្ហាញ និងរាប់ចំនួនលេខសេសទាំងអស់នៅក្នុងដែននោះ។
- **Edge Case:** ប្រសិនបើ Start ធំជាង End ត្រូវប្រាប់ឱ្យអ្នកប្រើបញ្ចូលដែនសារជាថ្មី។
- **Expected Output:** Input: Start=5, End=11 -> Output: 5, 7, 9, 11 (Count: 4)

លំហាត់ទី 52: ផលបូកលេខគូពី ១ ដល់ ១០០ (Sum of Even Numbers)

- **Business Logic:** គណនាផលបូកនៃរាល់លេខគូទាំងអស់ចាប់ពី ១ ដល់ ១០០ ដោយប្រើប្រាស់ Loop និងលក្ខខណ្ឌ $\% 2 == 0$ ឬបង្កើនបោះដំហាន Loop ម្តងត្រឹម ២ ជម្រៅ។
- **Edge Case:** ការបង្កើនបោះដំហានម្តង ២ ជម្រៅ ($i += 2$) មានប្រសិទ្ធភាពជាងការបែកលក្ខខណ្ឌរាល់ដង។
- **Expected Output:** Output: Total Sum of Even Numbers is 2550

លំហាត់ទី 53: គូររូបចតុកោណកែងប្រហោង (Hollow Rectangle Pattern)

- **Business Logic:** ទទួលតម្លៃជួរដេក (Rows) និងជួរឈរ (Cols)

រួចគូររូបចតុកោណកែងដែលប្រហោងខាងក្នុងដោយប្រើផ្កាយ `\*`។

- **Edge Case:** ផ្កាយត្រូវគូរតែនៅលើជួរដេកទី១, ជួរដេកចុងក្រោយ, ជួរឈរទី១ និងជួរឈរចុងក្រោយប៉ុណ្ណោះ។

- **Expected Output:** Input: Rows=3, Cols=4 -> Output:

```
****
```

```
*  *
```

```
****
```

លំហាត់ទី 54: Full Multiplication Matrix

- **Business Logic:** បង្កើតតារាងគុណ (Multiplication Table) ពីមេ ២ ដល់ ៩ បង្ហាញក្នុង Console តែម្តងជានិច្ច ប្រើ Grid។

- **Edge Case:** ប្រើប្រាស់ ` ` (Tab Escape Character) ដើម្បីឱ្យជួរឈរនីមួយៗត្រូវបានត្រួតពិនិត្យ។

- **Expected Output:** Output: A full grid showing tables from 2x1 up to 9x10

លំហាត់ទី 55: រកតួចែករួមធំបំផុត GCD (Greatest Common Divisor via Loop)

- **Business Logic:** បញ្ចូលលេខ ២ រួចស្វែងរកតួចែករួមធំបំផុតរបស់លេខទាំងពីរដោយប្រើវិធីសាស្ត្រ Loop ធម្មតា ឬ Euclidean Algorithm។

- **Edge Case:** ក្នុងករណីដែលមិនមានលេខរួមចែករួមគ្នា លេខនេះគឺជាលេខអវិជ្ជមានក៏ដោយ។

- **Expected Output:** Input: 12, 18 -> Output: GCD is 6

លំហាត់ទី 56: ពិនិត្យលេខ Armstrong (Armstrong Number Validation)

- **Business Logic:** ពិនិត្យថាលេខមួយជាលេខ Armstrong ឬទេ

(ផលបូកនៃស្វ័យគុណចំនួនខ្ទង់នៃតួអក្សរនីមួយៗស្មើនឹងតម្លៃដើម ឧ៖ $153 = 1^3 + 5^3 + 3^3$)។

- **Edge Case:** ដំបូងត្រូវគណនាចំនួនខ្ទង់សរុបនៃលេខនោះសិន មុននឹងគណនាស្វ័យគុណខ្ទង់នីមួយៗ។

- **Expected Output:** Input: 153 -> Output: 153 is an Armstrong Number

លំហាត់ទី 57: ការបញ្ចូលទិន្នន័យរហូតដល់សូន្យ (Sentinel-Controlled Loop)

- **Business Logic:** អនុញ្ញាតឱ្យអ្នកប្រើបញ្ចូលលេខរហូត តែបើគាត់បញ្ចូលលេខ 0 នោះកម្មវិធីនឹងឈប់ រួចបង្ហាញផលបូកលេខទាំងអស់ដែលបានបញ្ចូលពីមុន។

- **Edge Case:** លេខ 0 (Sentinel value) មិនត្រូវយកមកបូកបញ្ចូលក្នុងផលបូកចុងក្រោយឡើយ។

- **Expected Output:** Input: 5, 10, 2, 0 -> Output: Total Sum: 17

លំហាត់ទី 58: គណនាផលបូកស្វ័យគុណ (Sum of Squares Series)

- **Business Logic:** គណនាស៊េរីគណិតវិទ្យា $1^2 + 2^2 + 3^2 + \dots + N^2$ ។

- **Edge Case:** ត្រូវប្រើប្រាស់ប្រភេទដេតាវីយេបល `long` សម្រាប់អនុវត្តផលបូក បើសិនជាតម្លៃ N ធំ ដើម្បីការពារការកើតមាន Integer Overflow។

- **Expected Output:** Input: 3 -> Output: 1 + 4 + 9 = 14

លំហាត់ទី 59: រាប់ចំនួនខ្ទង់នៃលេខ (Digit Counter Loop)

- **Business Logic:** បញ្ចូលលេខគត់មួយ (ទោះជាប៉ុណ្ណាក៏ដោយ) រួចរាប់ថាមានប៉ុន្មានខ្ទង់ដោយប្រើប្រាស់ Loop ចែកនឹង ១០។

- **Edge Case:** ករណីពិសេស៖ ប្រសិនបើអ្នកប្រើបញ្ចូលលេខ ០ ផ្ទាល់ កម្មវិធីត្រូវតែឆ្លើយថាមាន ១ ខ្ទង់ (មិនមែន ០ ខ្ទង់ទេ)។

- **Expected Output:** Input: 4567 -> Output: Number of digits: 4

លំហាត់ទី 60: លំនាំលេខជណ្តើរ (Number Pyramid Grid)

• **Business Logic:** បង្កើតតួលេខជណ្តើរលេខជាន់ត្រូវតែកើន (ជួរទី១ ចេញលេខ ១, ជួរទី២ ចេញលេខ ១ ២, ជួរទី៣ ចេញលេខ ១ ២ ៣)។

• **Edge Case:** Loop ខាងក្នុងត្រូវប្រើតម្លៃចាប់ផ្តើម និងបញ្ចប់អាស្រ័យលើ Loop ខាងក្រៅ។

• **Expected Output:** Input: 3 -> Output:

```
1
1 2
1 2 3
```

កម្រិតទី ៤: Array និងបណ្តុំទិន្នន័យ (Arrays, Strings & Functions)

លំហាត់ទី 61: បញ្ជីឈ្មោះ Scholars (Array Storage and Iteration)

• **Business Logic:** រក្សាទុកឈ្មោះសិស្ស ១០ នាក់ក្នុង Array នៃ String រួចប្រើ For-Each Loop ដើម្បីបង្ហាញឈ្មោះទាំងអស់មកវិញ។

• **Edge Case:** ប្រសិនបើទីតាំងណាមួយក្នុង Array មានតម្លៃជា 'null' ក្នុងកំឡុងពេល Skip ចោលមិនបង្ហាញដើម្បីកុំឱ្យ NullPointerException។

• **Expected Output:** Output: List of 10 students displayed sequentially.

លំហាត់ទី 62: ស្វែងរកឈ្មោះក្នុងប្រព័ន្ធ (Linear Search Array Algorithm)

• **Business Logic:** ឆែកមើលថាតើឈ្មោះដែលវាយបញ្ចូល មានផ្ទុកនៅក្នុង Array ឈ្មោះដែលមានស្រាប់ឬអត់ រួចបង្ហាញទីតាំង Index របស់វា។

• **Edge Case:** ការប្រៀបធៀប String ត្រូវប្រើប្រាស់ Method `.equalsIgnoreCase()` ជំនួសឱ្យ `==`។

• **Expected Output:** Input: 'Sok' -> Output: Found at index 2 | Input: 'Dara' -> Not Found

លំហាត់ទី 63: ប្តូរតម្លៃចុងសងខាង Array (Array Element Swapping)

• **Business Logic:** សរសេរកូដប្តូរតម្លៃរវាង Element ទីមួយ (Index 0) និង Element ចុងក្រោយគេបង្អស់ (`array.length - 1`) នៅក្នុង Array មួយ។

• **Edge Case:** ត្រូវបង្កើតអថេរបណ្តោះអាសន្ន (Temporary variable) ដើម្បីរក្សាទុកតម្លៃកុំឱ្យបាត់បង់ពេលប្តូរ។

• **Expected Output:** Input: [10, 20, 30, 40] -> Output: [40, 20, 30, 10]

លំហាត់ទី 64: រកពិន្ទុមធ្យមភាគថ្នាក់ (Class Average via Array)

• **Business Logic:** រក្សាទុកពិន្ទុសិស្ស ២០ នាក់ក្នុង Array រួចគណនារកពិន្ទុមធ្យមភាគរបស់ថ្នាក់ទាំងមូល។

• **Edge Case:** ការគណនាមធ្យមភាគ ត្រូវបំប្លែងធុលបូកទៅជាប្រភេទ `'double'` មុននឹងចែកនឹងប្រវែង Array ដើម្បីកុំឱ្យបាត់តម្លៃក្បៀស។

• **Expected Output:** Input: Array of 20 integers -> Output: Class Average: 78.45

លំហាត់ទី 65: តម្រៀបពិន្ទុសិស្ស (Array Bubble Sort Implementation)

• **Business Logic:** តម្រៀបលេខនៅក្នុង Array ពីតូចទៅធំដោយសរសេរ Algorithm Bubble Sort ផ្ទាល់ដោយមិនប្រើ `Arrays.sort()`។

• **Edge Case:** Nested Loops ត្រូវតម្រៀប និងប្រៀបធៀបធាតុជិតខាងគ្នា រួចធ្វើការ Swap បើតម្លៃធាតុខាងមុខធំជាងធាតុខាងក្រោយ។

• **Expected Output:** Input: [5, 1, 4, 2] -> Output: [1, 2, 4, 5]

លំហាត់ទី 66: រាប់តួអក្សរមិនគិតចន្លោះ (String Character Counter)

- **Business Logic:** ទទួល String មួយពីអ្នកប្រើប្រាស់ រួចរាប់ចំនួនតួអក្សរទាំងអស់ដោយមិនរាប់បញ្ចូលចន្លោះទំនេរ (Spaces) ឡើយ។
- **Edge Case:** ប្រើប្រាស់ Loop និង Method `charAt(i)` រួចត្រួតពិនិត្យតួអក្សរនីមួយៗ។
- **Expected Output:** Input: 'Hello ISTAD' -> Output: Character Count: 10

លំហាត់ទី 67: រកតួអក្សរដដែលក្នុង Array (Find Duplicate Array Elements)

- **Business Logic:** ស្វែងរក និងបង្ហាញលេខដែលមានតួអក្សរដដែលក្នុង Array មួយ (លេខឡើងលើសពីម្តង)។
- **Edge Case:** ធានាដែលរកឃើញតួអក្សរហើយ មិនត្រូវបោះពុម្ពបង្ហាញដល់លទ្ធផលនោះទេ។
- **Expected Output:** Input: [2, 4, 5, 2, 6, 4] -> Output: Duplicates found: 2, 4

លំហាត់ទី 68: ផលបូកម៉ាទ្រីស ២ វិមាត្រ (2D Array Matrix Addition)

- **Business Logic:** បង្កើត Matrix ទំហំ 3x3 ចំនួនពីរ រួចធ្វើការបូកបញ្ចូលគ្នាតាមទីតាំងនីមួយៗ ហើយរក្សាទុកក្នុង Matrix ទីបី។
- **Edge Case:** Matrix ទាំងពីរត្រូវតែមានទំហំដូរគ្នា និងជួរឈរប៉ុនគ្នាពិតប្រាកដ ទើបអាចបូកបាន។
- **Expected Output:** Output: A new 3x3 matrix representing the sum of Matrix A and B

លំហាត់ទី 69: Method ប្តូរប្រាក់ប្រើឡើងវិញបាន (Reusable Currency Conversion Method)

- **Business Logic:** បង្កើត Method ឈ្មោះ `convertToRiel(double usd)` ដែលទទួលតម្លៃដុល្លាររួចត្រឡប់ (return) តម្លៃជាលុយរៀលមកវិញ ដើម្បីហៅប្រើក្នុង Main។
- **Edge Case:** Method ត្រូវមាន Access Modifier ជា `public static` ដើម្បីអាចហៅប្រើបានក្នុង Class តែមួយដោយមិនចាប់ផ្តើម Object។
- **Expected Output:** Calling: `convertToRiel(10.0)` -> Returns: 41000.0

លំហាត់ទី 70: ពិនិត្យពាក្យចង្កៀង (String Palindrome Checker)

- **Business Logic:** ពិនិត្យមើលពាក្យមួយ (ឧទាហរណ៍: 'RADAR') ថាទោះបីជាអានចេញក្រោយក៏នៅតែមានន័យ និងតួអក្សរដូចដើមឬទេ។
- **Edge Case:** ក្នុងត្រូវប្រៀបធៀបតួអក្សរពីមុខ និងពីក្រោយខិតចូលគ្នា ('i' កើន និង 'j' ថយចុះ)។
- **Expected Output:** Input: 'kayak' -> Output: Palindrome | Input: 'java' -> Output: Not Palindrome

លំហាត់ទី 71: រកតម្លៃតូចបំផុត និងធំបំផុតក្នុង Array (Min/Max Array Elements)

- **Business Logic:** ស្វែងរកតម្លៃតូចបំផុត (Min) និងតម្លៃធំបំផុត (Max) នៅក្នុង Array តែមួយដោយប្រើការរត់ Loop តែម្តងគត់ (Single Pass)។
- **Edge Case:** ចាប់ផ្តើមដោយតម្លៃ Min និង Max ស្មើនឹងធាតុដំបូងគេបង្អស់នៃ Array។
- **Expected Output:** Input: [12, 4, 56, 1, 9] -> Output: Min: 1, Max: 56

លំហាត់ទី 72: បញ្ចៀសធាតុ Array (Array In-Place Reversal)

- **Business Logic:** សរសេរកម្មវិធីដើម្បីត្រឡប់ជើង Array ពីក្រោយមកមុខវិញដោយប្តូរទីតាំងនៅក្នុង Array ដើមផ្ទាល់ (In-place reversal) មិនបង្កើត Array ថ្មីទេ។
- **Edge Case:** Loop ត្រូវរត់ត្រឹមតែពាក់កណ្តាលនៃប្រវែង Array ប៉ុណ្ណោះ (`i < array.length / 2`)។
- **Expected Output:** Input: [1, 2, 3, 4] -> Output: [4, 3, 2, 1]

លំហាត់ទី 73: គណនាមធ្យមភាគ Diagonal នៃ Matrix (Matrix Diagonal Statistics)

- **Business Logic:** គណនាផលបូក និងមធ្យមភាគនៃតម្លៃដែលស្ថិតនៅលើបន្ទាត់ទ្រូង (Main Diagonal) នៃ 2D Array ទំហំ $N \times N$ ។
- **Edge Case:** ធានានៅលើបន្ទាត់ទ្រូងគឺជាតម្លៃដែលមានលេខជួរដេក និងជួរឈរស្មើគ្នា (`matrix[i][i]`)។
- **Expected Output:** Output: Sum and average of elements where row index equals column index

លំហាត់ទី 74: បំបែកពាក្យក្នុងប្រយោគ (String Word Tokenizer and Counter)

- **Business Logic:** ទទួលយកប្រយោគមួយ រួចបំបែកវាទៅជាពាក្យដាច់ដោយឡែកពីគ្នាដោយផ្អែកលើចន្លោះ (Space) រួចរាប់ចំនួនពាក្យសរុប។
- **Edge Case:** ក្នុងត្រូវប្រើប្រាស់ Method `.split("\\s+")` ដើម្បីទប់ស្កាត់ករណីមានដកហូតចន្លោះនៅចន្លោះពាក្យ។
- **Expected Output:** Input: 'Welcome to Java' -> Output: Word Count: 3

លំហាត់ទី 75: បំប្លែងតួអក្សរក្នុងទម្រង់ជាអក្សរធំ (Sentence Capitalization Utility)

- **Business Logic:** ទទួល String ជាឈ្មោះអក្សរតូចទាំងអស់ រួចបំប្លែងតួអក្សរដំបូងនៃពាក្យនីមួយៗឱ្យទៅជាអក្សរធំ (Title Case)។
- **Edge Case:** ត្រូវប្រើប្រាស់ `StringBuilder` ដើម្បីផ្គុំតួអក្សរឡើងវិញក្រោយបំប្លែងរួច។
- **Expected Output:** Input: 'cambodia tech' -> Output: 'Cambodia Tech'

លំហាត់ទី 76: ការលុបតម្លៃចេញពី Array (Array Element Deletion & Shifting)

- **Business Logic:** បង្កើត Array មួយ រួចលុប Element នៅទីតាំង Index ណាមួយចោលដោយរំកិល (Shift) Element ខាងក្រោយមកបំពេញចន្លោះ។
- **Edge Case:** ធានាថាប្រើប្រាស់លក្ខណៈរបស់ Array ក្រោយពេលរំកិលរួច ត្រូវកំណត់តម្លៃលំដាប់ដើមទៅជា ០។
- **Expected Output:** Input: [10, 20, 30, 40], Delete index 1 -> Output: [10, 30, 40, 0]

លំហាត់ទី 77: Merge Two Sorted Arrays

- **Business Logic:** យក Array ចំនួនពីរដែលមានទិន្នន័យខុសគ្នា មកបញ្ចូលគ្នាបង្កើតបានជា Array ធំមួយដែលមានទំហំស្មើនឹងផលបូកប្រវែង Array ទាំងពីរ។
- **Edge Case:** Array ថ្មីត្រូវមានទំហំធំល្មមដើម្បីផ្ទុកធាតុទាំងអស់របស់ Array ទាំងពីររួមគ្នា។
- **Expected Output:** Input: [1, 2] and [3, 4] -> Output: [1, 2, 3, 4]

លំហាត់ទី 78: Method គណនាការប្រាក់ Overloading (Method Overloading Architecture)

- **Business Logic:** បង្កើត Method ឈ្មោះ `calculateInterest()` ចំនួនពីរ៖ មួយទទួល Parameter រយៈពេលជាឆ្នាំ (int) និងមួយទៀតទទួលរយៈពេលជាខែ (double)។
- **Edge Case:** នេះជាការអនុវត្តគំនិត Polymorphism (Compile-time) តាមរយៈការធ្វើ Method Overloading។
- **Expected Output:** Overloads: `calculateInterest(int years)` vs `calculateInterest(double months)`

លំហាត់ទី 79: រាប់ចំនួនព្យញ្ជនៈ និងស្រៈក្នុងអក្សរអង់គ្លេស (Vowel and Consonant Counter)

- **Business Logic:** ទទួល String ភាសាអង់គ្លេសមួយ រួចរាប់ថាមានស្រៈ (A, E, I, O, U) ប៉ុន្មាន និងព្យញ្ជនៈប៉ុន្មាន។
- **Edge Case:** មុននឹងបំប្លែង String ទាំងមូលទៅជាអក្សរតូចសិន ដើម្បីងាយស្រួលប្រៀបធៀប។
- **Expected Output:** Input: 'Java' -> Output: Vowels: 2, Consonants: 2

លំហាត់ទី 80: ការត្រួតពិនិត្យភាពខ្លាំងនៃ Password (Password Strength Validator)

- **Business Logic:** បង្កើត Method ពិនិត្យ String ជា Password បើវាមានប្រវែងលើសពី ៨ ខ្ទង់ មានទាំងអក្សរធំ អក្សរតូច និងលេខ ទើបចាត់ទុកថា 'Strong' (return boolean)។
- **Edge Case:** ត្រូវប្រើ Loop រត់ឆែកមើលតួអក្សរម្តងមួយៗ និងប្រើប្រាស់ `Character.isDigit()`, `Character.isUpperCase()`។
- **Expected Output:** Input: 'Abc12345' -> Output: true (Strong)

កម្រិតទី ៥: កម្រិតខ្ពស់ និង Logic ស្មុគស្មាញ (Algorithms & OOP Concept)

លំហាត់ទី 81: ប្រព័ន្ធគ្រប់គ្រងស្តុកទំនិញ (OOP Inventory System)

- **Business Logic:** បង្កើត Class Product មាន Attributes (id, name, price, quantity) និង Method ឈ្មោះ `sellProduct(int qty)` សម្រាប់ដកកាត់ស្តុកនៅពេលមានការលក់។
- **Edge Case:** ប្រសិនបើចំនួនបញ្ជាទិញ (qty) ធំជាងចំនួនទំនិញដែលមានក្នុងស្តុក ត្រូវបង្ហាញសារបដិសេធការលក់ (Insufficient Stock)។
- **Expected Output:** Product stock=10 -> sellProduct(12) -> Output: Stock insufficient.

លំហាត់ទី 82: ប្រព័ន្ធចុះឈ្មោះ Scholars ចូលរៀន (Student Course Enrollment OOP)

- **Business Logic:** បង្កើត Class Scholar និង Class Course។ នៅក្នុង Class Course ត្រូវមាន Array នៃ Scholar រួចបង្កើត Method `enroll(Scholar s)` ដើម្បីបន្ថែមសិស្សចូលរៀន។
- **Edge Case:** វគ្គសិក្សានីមួយៗត្រូវកំណត់ចំនួនសិស្សអតិបរមា (ឧ: ៣០នាក់) បើពេញមិនឱ្យចុះឈ្មោះទៀតទេ។
- **Expected Output:** Course capacity=30 -> enrollment 31st student -> Output: Course is full.

លំហាត់ទី 83: គណនាពន្ធលើប្រាក់បៀវត្សខ្មែរ (Cambodian Tax on Salary Calculator)

- **Business Logic:** បង្កើត Logic គណនាពន្ធលើប្រាក់បៀវត្ស (Tax on Salary) តាមភាគរយ (0%, 5%, 10%, 15%, 20%)។
- **Edge Case:** ត្រូវដកប្រាក់កាត់បន្ថយការជាប់ពន្ធសម្រាប់សមាជិកក្នុងបន្ទុក ម្នាក់ ១៥០,០០០ រៀល សិនមុននឹងយកទៅគិតពន្ធ។
- **Expected Output:** Input: Salary=2,000,000 KHR, 2 Kids -> Taxable Salary reduction applied -> Calculate step tax.

លំហាត់ទី 84: កម្មវិធីគ្រប់គ្រងបណ្ណាល័យ (Library Book Borrowing Architecture)

- **Business Logic:** បង្កើតប្រព័ន្ធ Object សម្រាប់តាមដានសៀវភៅ ស្ថានភាពខ្ចី (isBorrowed) និងការផាកពិន័យទឹកប្រាក់ (\$0.50 ក្នុងមួយថ្ងៃ) បើហួសថ្ងៃកំណត់សង។
- **Edge Case:** សៀវភៅដែលត្រូវបានខ្ចីរួចហើយ មិនអាចអនុញ្ញាតឱ្យអ្នកផ្សេងទៀតខ្ចីបានទៀតទេ លុះត្រាតែត្រូវបានសងចូលបណ្ណាល័យវិញសិន។
- **Expected Output:** Book isBorrowed=true -> try borrow -> Output: Book is currently unavailable.

លំហាត់ទី 85: ម៉ាស៊ីនគិតលេខ String Parser (Expression Math String Parser)

- **Business Logic:** កម្មវិធីទទួលទិន្នន័យជា String ដូចជា '150 + 250' រួចប្រើ Logic បំបែក (Parsing) ញែកយកលេខ និងសញ្ញាប្រមាណវិធីមកគណនារកលទ្ធផលពិត។
- **Edge Case:** ក្នុងត្រូវចេះប្រើប្រាស់ Method `indexOf()` ឬ `split()` រួចបំបែកពី String ទៅជា Integer វិញតាមរយៈ `Integer.parseInt()`។
- **Expected Output:** Input: '100 * 5' -> Output: Result = 500

លំហាត់ទី 86៖ ប្រព័ន្ធកកសំបុត្រឡានក្រុងខេត្ត (Bus Seat Reservation System)

- **Business Logic:** ប្រើ 2D Array នៃ Object ប្រភេទ Seat តំណាងឱ្យកៅអីឡានក្រុង (ជួរដេក និងជួរឈរ) រួចធ្វើ Logic ពិនិត្យមើលកៅអីទំនេរ, កកកៅអី, និងលុបការកក។
- **Edge Case:** កៅអីដែលបានកករួច (isBooked == true) ត្រូវការពារមិនឱ្យអ្នកផ្សេងមកកកត្រួតពីលើបានឡើយ។
- **Expected Output:** Seat [0][2] booked -> try book again -> Output: Seat already taken.

លំហាត់ទី 87៖ Logic បំបែកក្រដាសប្រាក់អាប់ (Cash Denomination Optimization)

- **Business Logic:** ប្រសិនបើត្រូវអាប់លុយឱ្យអតិថិជនចំនួន N រៀល តើប្រព័ន្ធត្រូវបំបែកជាប្រាក់ក្រដាស ១០០០០៛, ៥០០០៛, ១០០០៛, និង ៥០០៛ ប៉ុន្មានសន្លឹក ដើម្បីប្រើចំនួនសន្លឹកតិចបំផុត? (Greedy Algorithm)
- **Edge Case:** ត្រូវប្រើប្រាស់ Loop ឬវិធីចែកដាច់យកចំនួនសន្លឹក និងវិធីរកសំណល់ '%' ដើម្បីកាត់ទឹកប្រាក់ចេញជាលំដាប់ពីក្រដាសធំទៅក្រដាសតូច។
- **Expected Output:** Input: 16500 Riel -> Output: 10000x1, 5000x1, 1000x1, 500x1

លំហាត់ទី 88៖ ការបង្កើត Leaderboard សិស្ស (Scholar Academic Honor Roll Sorting)

- **Business Logic:** ទទួលយក Array នៃ Object Scholar រួចតម្រៀបពួកគេតាមមធ្យមភាគពិន្ទុពីខ្ពស់ទៅទាបបំផុតដើម្បីបង្កើតតារាងចំណាត់ថ្នាក់ Honor Roll (Leaderboard)។
- **Edge Case:** ក្នុងករណីមានសិស្សពីរនាក់មានពិន្ទុស្មើគ្នា (Tie) ត្រូវតម្រៀបតាមលំដាប់អក្សរនៃឈ្មោះរបស់ពួកគេបន្ត (Alphabetical tie-breaker)។
- **Expected Output:** Input: Scholars Array -> Sorted by GPA DESC, then by Name ASC.

លំហាត់ទី 89៖ Simulate PaaS Micro-Billing (Cloud Infrastructure Resource Metering)

- **Business Logic:** គណនាតម្លៃសេវា Cloud Infrastructure ដោយគិតលុយតាមទំហំ RAM/CPU និងចំនួនវិនាទីដែល Container ដំណើរការជាក់ស្តែង។
- **Edge Case:** ត្រូវប្រើប្រាស់ប្រភេទ 'double' សម្រាប់តម្លៃទឹកប្រាក់មីក្រូ ដើម្បីកុំឱ្យបាត់បង់តម្លៃកាក់តូចៗពេលគណនីពេលវេលាជារន្ទិះ។
- **Expected Output:** Formula: Cost = (RAM * 0.00001\$ + CPU * 0.00002\$) * SecondsUsed

លំហាត់ទី 90៖ Search Engine និង Keyword Highlighting (Text Indexing Engine)

- **Business Logic:** សរសេរ Algorithm ស្វែងរកពាក្យគន្លឹះនៅក្នុងអត្ថបទវែងមួយ រួចបង្ហាញទីតាំង Index ទាំងអស់នៃពាក្យនោះ និងគណនាចំនួនដងដែលវាបានលេចឡើង។
- **Edge Case:** ត្រូវប្រើវង្វិលជុំ 'while' រួមជាមួយ Method '.indexOf(keyword, fromIndex)' ដើម្បីស្វែងរកបន្តបន្ទាប់គ្នាទាល់តែអស់។
- **Expected Output:** Input: text='java is good, learn java', keyword='java' -> Output: Found at indices: 0, 19. Count: 2

លំហាត់ទី 91៖ ប្រព័ន្ធគ្រប់គ្រងគណនេយ្យធនាគារ (Encapsulation and Security Model)

- **Business Logic:** បង្កើត Class BankAccount ដោយលាក់បាំងសមតុល្យលុយ (private balance) ហើយអនុញ្ញាតឱ្យកែប្រែតាមតែ Method 'deposit()' និង 'withdraw()' ដែលមាន Logic ត្រួតពិនិត្យសុវត្ថិភាពប៉ុណ្ណោះ។
- **Edge Case:** Method 'withdraw()' ត្រូវហាមឃាត់ដាច់ខាតមិនឱ្យដកលុយហួសសមតុល្យដែលមានស្រាប់ (No negative balance allowed)។
- **Expected Output:** Balance=100 -> withdraw(150) -> Output: Transaction rejected. Insufficient balance.

លំហាត់ទី 92: ប្រព័ន្ធគណនាប្រាក់ខែបុគ្គលិក (Employee Inheritance Hierarchy)

- **Business Logic:** បង្កើត Base Class Employee និង Subclasses FullTimeEmployee (គិតប្រាក់ខែថេរ + ប្រាក់លើកទឹកចិត្ត) និង PartTimeEmployee (គិតតាមម៉ោងធ្វើការជាក់ស្តែង)។
- **Edge Case:** ប្រើប្រាស់គំនិត Inheritance ដើម្បីទាញយក Attribute រួម (id, name) និងប្រើប្រាស់ Method Overriding សម្រាប់មុខងារ `calculateSalary()`។
- **Expected Output:** FullTime -> returns base+bonus | PartTime -> returns hoursWorked * hourlyRate

លំហាត់ទី 93: Polygon Area Calculator (Polymorphic Shape Architecture)

- **Business Logic:** បង្កើត Abstract Class Shape និងមាន abstract method `calculateArea()` រួចឱ្យ Class Circle, Rectangle, និង Square យកទៅ Override តាមរូបមន្តរៀងៗខ្លួន។
- **Edge Case:** នេះអនុញ្ញាតឱ្យយើងបង្កើត Array នៃ Shape រួចរត់ Loop គណនាផ្ទៃក្រឡារូបរាងទាំងអស់បញ្ចូលគ្នាបានយ៉ាងងាយស្រួល (Dynamic Polymorphism)។
- **Expected Output:** Shape[] shapes = { new Circle(5), new Rectangle(4,5) }; -> Loop and calculate area.

លំហាត់ទី 94: ប្រព័ន្ធគ្រប់គ្រងការបញ្ជាទិញ (E-Commerce Promo Code Order Processing)

- **Business Logic:** បង្កើត Class Order ដែលផ្ទុក Array នៃ OrderItem នីមួយៗ រួចគណនាតម្លៃសរុប រួមទាំងការគិតបញ្ចុះតម្លៃតាមប្រមូលកូដ (Promo Code) ដូចជា 'ISTAD_PROMO' បញ្ចុះតម្លៃ ១៥%។
- **Edge Case:** ការបញ្ចុះតម្លៃត្រូវអនុវត្តទៅលើតម្លៃសរុបក្រោយពេលបូកបញ្ចូលតម្លៃ OrderItem ទាំងអស់រួចរាល់តែប៉ុណ្ណោះ។
- **Expected Output:** Total=\$100 -> Promo='ISTAD_PROMO' -> Final Total = \$85

លំហាត់ទី 95: ការវិភាគទិន្នន័យអាកាសធាតុ (Matrix Stats & Temperature Analytics)

- **Business Logic:** ប្រើ 2D Array រក្សាទុកសីតុណ្ហភាពប្រចាំម៉ោង រយៈពេល ៧ ថ្ងៃ រួចរកចំណាត់ថ្នាក់សីតុណ្ហភាពក្តៅជាងគេបំផុត និងមធ្យមភាគសីតុណ្ហភាពប្រចាំសប្តាហ៍។
- **Edge Case:** ត្រូវប្រើ Nested Loops រត់កាត់គ្រប់ធាតុទាំងអស់ក្នុង Matrix ដើម្បីរកតម្លៃអតិបរមា (Max) សកល។
- **Expected Output:** Output: Maximum temperature recorded was 39°C on Day 3.

លំហាត់ទី 96: កម្មវិធីផ្ទៀងផ្ទាត់ទម្រង់ទិន្នន័យ (Cambodian Identity & Phone Pattern Validator)

- **Business Logic:** បង្កើតប្រព័ន្ធពិនិត្យមើលអត្ថបទដែលបញ្ចូល ថាតើវាជាទម្រង់ Email ត្រឹមត្រូវ ឬជាលេខទូរស័ព្ទប្រព័ន្ធកម្ពុជាត្រឹមត្រូវ (ចាប់ផ្តើមដោយលេខ ០ និងមានប្រវែង ៩ ទៅ ១០ ខ្ទង់) ដោយប្រើប្រាស់ Logic ឆែក Char ម្តងមួយៗ។
- **Edge Case:** មិនត្រូវប្រើ library ឬ Regular Expressions (Regex) ឡើយ គឺត្រូវប្រើ Loop/Condition logic សុទ្ធ។
- **Expected Output:** Input: '012345678' -> Output: Valid Cambodian Phone Number Format.

លំហាត់ទី 97: ប្រព័ន្ធចុះឈ្មោះចូលប្រើប្រាស់ (In-Memory User Authentication System)

- **Business Logic:** បង្កើតប្រព័ន្ធរក្សាទុក Username និង Password ក្នុង Arrays រួចធ្វើមុខងារចុះឈ្មោះ (Register) និងមុខងារ Login ដែលផ្ទៀងផ្ទាត់ទិន្នន័យ។
- **Edge Case:** ពេលចុះឈ្មោះ ត្រូវឆែកមើលថា Username នោះមានអ្នកប្រើប្រាស់រួចហើយឬនៅ (Username uniqueness check)។
- **Expected Output:** Username 'admin' exists -> try register 'admin' -> Output: Username already taken.

លំហាត់ទី 98: សេវាកម្មកក់សណ្ឋាគារនៅសៀមរាប (Siem Reap Hotel Room Booking Core)

- **Business Logic:** បង្កើត Class Hotel ដែលមាន Array នៃ Room Object។ សរសេរ Logic សម្រាប់ឆែកមើលបន្ទប់ទំនេរ (Check Availability), ធ្វើការកក់ (Book), និងការចាកចេញ (Check-out)។
- **Edge Case:** នៅពេល Check-out ស្ថានភាពបន្ទប់ត្រូវតែផ្លាស់ប្តូរមកជាទំនេរវិញ (isAvailable = true) ដើម្បីឱ្យអ្នកដទៃអាចកក់បន្តបាន។
- **Expected Output:** Room 101 checked out -> isAvailable becomes true automatically.

លំហាត់ទី 99: កម្មវិធីតាមដានការចំណាយប្រចាំថ្ងៃ (Smart Personal Budget Tracker)

- **Business Logic:** បង្កើតកម្មវិធីដែលអាចឱ្យអ្នកប្រើប្រាស់កត់ត្រាការចំណាយតាមប្រភេទ (Food, Transport, Education) រួចបង្ហាញរបាយការណ៍សង្ខេបថាប្រភេទណាដែលចំណាយច្រើនជាងគេ។
- **Edge Case:** ត្រូវប្រើប្រាស់ Array ដើម្បីបូកសរុបទឹកប្រាក់ដាច់ដោយឡែកពីគ្នាតាមប្រភេទចំណាយនីមួយៗ (Category aggregation)។
- **Expected Output:** Output: Highest expense category this month: Food (\$120.00)

លំហាត់ទី 100: Algorithm ស្វែងរកឯកសារចម្លង (Simple Plagiarism & Similarity Detector)

- **Business Logic:** ទទួល String ពីប្រយោគ រួចប្រៀបធៀបរកមើលភាគរយនៃភាពដូចគ្នា (Similarity Percentage) នៃពាក្យនៅក្នុងប្រយោគទាំងពីរនោះ។
- **Edge Case:** Logic ត្រូវគណនាដោយ: (ចំនួនពាក្យដែលដូចគ្នា / ចំនួនពាក្យសរុបនៃប្រយោគដែលវែងជាង) * 100។
- **Expected Output:** Input: 'i love java', 'i code java' -> Common words: 'i', 'java' (2)
-> Similarity: $(2 / 3) * 100 = 66.6\%$